Relatório do Projeto — Mini-Framework de Jogos de Cartas

Disciplina: Métodos Avançados de Programação (MAP) — 2026.1 Projeto: Mini-Framework Orientado a Objetos para Jogos de Cartas Tecnologias: Java 17+, Maven, JUnit 4, Javadoc

---

1. Descrição Geral da Arquitetura

O projeto consiste em um mini-framework reutilizável para jogos de cartas, organizado em dois pacotes principais com responsabilidades bem definidas:

1.1 Pacote `com.map.framework` (Componente Reutilizável)

Este pacote contém o núcleo genérico do framework, 100% independente de qualquer jogo específico. Ele é composto por:

Subpacote	Responsabilidade	Classes principais
`card`	Abstração de cartas e baralhos	`Carta` (abstrata), `Baralho` (abstrata), `BaralhoTradicional`, `CartaTradicional`, `Naipe`, `EmbaralhadorStrategy`, `EmbaralhadorAleatorio`, `SemEmbaralhamento`
`player`	Abstração de jogadores e estratégias	`Jogador`, `Mao`, `EstrategiaJogador`, `EstrategiaHumanaConsole`, `EstrategiaIAAleatoria`, `AcaoJogador`
`game`	Orquestração de partidas	`Partida` (abstrata), `PartidaSimples`, `RegrasJogo`, `PartidaListener`, `PartidaConsoleListener`, `PartidaEvento`
`exception`	Tratamento de erros	`BaralhoVazioException`, `CartaNaoEncontradaException`, `JogoInvalidoException`, `MaoCheiaException`
(raiz)	API pública	`GameFacade`

1.2 Pacote `com.map.games` (Aplicações Cliente)

Este pacote contém os jogos concretos que utilizam o framework:

Jogo	Classe principal	Características
Blackjack (21)	`com.map.games.blackjack.MainBlackjack`	Jogo principal; humano vs IA; 1-6 baralhos; ações HIT/STAND
Super Trunfo	`com.map.games.supertrunfo.MainSuperTrunfo`	Aplicação alternativa; 32 cartas de carros; 5 atributos

1.3 Fluxo de Execução

```
Cliente (jogo) → GameFacade → Partida (Template Method)
                                  ├── Baralho (Factory Method)
                                  ├── RegrasJogo (Strategy)
                                  ├── EstrategiaJogador (Strategy)
                                  └── PartidaListener (Observer)
```

---

2. Justificativa das Principais Decisões de Projeto

2.1 Separação Framework / Aplicações

Decisão: Separar o código genérico (`com.map.framework`) do código específico dos jogos (`com.map.games`).

Justificativa: Esta é a decisão arquitetural mais importante. Ela garante que o framework seja reutilizável e extensível sem modificação. Um novo jogo (ex: Uno) pode ser criado apenas implementando as interfaces e estendendo as classes abstratas, sem alterar nenhuma linha do framework. Isso atende ao requisito de separação entre código da solução e código das aplicações clientes.

2.2 API Pública via Facade

Decisão: Criar a classe `GameFacade` como ponto único de entrada.

Justificativa: O padrão Facade simplifica o uso do framework, escondendo a complexidade interna. O cliente não precisa conhecer `Partida`, `RegrasJogo`, `Baralho`, etc. — basta chamar `GameFacade.criarPartida()` e `GameFacade.executarPartida()`. Isso atende ao requisito de API pública claramente definida.

2.3 Tipagem Genérica Forte

Decisão: Usar generics em `Partida<B extends Baralho, R extends RegrasJogo<B>>`.

Justificativa: A tipagem genérica garante segurança de tipos em tempo de compilação. O compilador verifica que o baralho `B` e as regras `R` são compatíveis, evitando erros em tempo de execução. Isso torna o framework mais robusto e auto-documentado.

2.4 Encapsulamento das Coleções

Decisão: Retornar coleções imutáveis via `Collections.unmodifiableList()`.

Justificativa: As coleções internas (`Mao.cartas`, `Baralho.cartas`, `Partida.jogadores`) são protegidas contra modificação externa. O acesso é feito apenas através de métodos controlados (`adicionarCarta`, `comprarCarta`, etc.). Isso atende ao requisito de encapsulamento das coleções internas.

2.5 Tratamento de Exceções

Decisão: Criar exceções customizadas no pacote `exception`.

Justificativa: Exceções específicas (`BaralhoVazioException`, `MaoCheiaException`, etc.) tornam o código mais expressivo e facilitam o tratamento de erros pelos jogos. Todas estendem `RuntimeException`, simplificando o uso sem obrigar o cliente a capturar exceções desnecessariamente.

---

3. Padrões de Projeto GoF Utilizados

3.1 Factory Method — Criação de Cartas

Onde: `Baralho.criarCartas()` (método abstrato)

Descrição: O método abstrato `criarCartas()` é o Factory Method. Cada subclasse de `Baralho` implementa a criação das suas cartas específicas:

- `BaralhoTradicional` → cria 52 cartas (Blackjack)
- `BaralhoSuperTrunfo` → cria 32 cartas de carros

Benefício: O framework não precisa conhecer as cartas concretas. A criação é delegada às subclasses, permitindo novos tipos de baralho sem modificar o framework.

3.2 Strategy — Comportamento Intercambiável (×3)

Onde: `EmbaralhadorStrategy`, `EstrategiaJogador`, `RegrasJogo<B>`

Descrição: O padrão Strategy é aplicado em três pontos:

1. `EmbaralhadorStrategy` — algoritmos de embaralhamento intercambiáveis (`EmbaralhadorAleatorio`, `SemEmbaralhamento`)
2. `EstrategiaJogador` — estratégias de decisão do jogador (`EstrategiaHumanaConsole`, `EstrategiaIAAleatoria`)
3. `RegrasJogo<B>` — regras específicas de cada jogo (`BlackjackRegras`, `SuperTrunfoRegras`)

Benefício: O comportamento pode ser trocado em tempo de execução sem modificar as classes que o utilizam. O `Jogador` delega para `EstrategiaJogador`; a `Partida` delega para `RegrasJogo`.

3.3 Template Method — Fluxo da Partida

Onde: `Partida.jogarPartida()`

Descrição: O método `jogarPartida()` é final (não pode ser sobrescrito) e define o esqueleto fixo do fluxo:

1. `inicializarPartida()` — embaralha e zera estado
2. `distribuirCartasIniciais()` — distribui as cartas
3. `rodarTurnos()` — loop de turnos
4. `finalizarPartida()` — define o vencedor

Os hooks (`aposDistribuicao`, `antesDaRodada`, `aposRodada`) permitem que as regras personalizem o fluxo.

Benefício: O fluxo é padronizado para todos os jogos, mas os detalhes são delegados às regras específicas.

3.4 Observer — Notificação de Eventos

Onde: `Partida` (Subject) + `PartidaListener` (Observer)

Descrição: A `Partida` mantém uma lista de `PartidaListener` e notifica-os sobre eventos

(PARTIDA_INICIADA, RODADA_FINALIZADA, CARTA_JOGADA, JOGADOR_ELIMINADO, etc.). O PartidaConsoleListener é uma implementação concreta que imprime os eventos no console.

Benefício: A partida não depende do console (ou de qualquer saída específica). Novas saídas (GUI, logs, JSON) podem ser adicionadas criando novos listeners, sem modificar a Partida.

---

4. Princípios GRASP Aplicados

Princípio	Aplicação no projeto
Creator	O Baralho cria suas próprias cartas (Factory Method). A Partida cria as notificações de eventos.
Information Expert	O Baralho conhece suas cartas (compra, embaralha). A Partida conhece o fluxo e os jogadores. As RegrasJogo conhecem as regras.
Low Coupling	Os jogos dependem de interfaces (RegrasJogo, EstrategiaJogador), não de classes concretas. A GameFacade reduz o acoplamento com o cliente.
High Cohesion	Os pacotes card, player, game têm responsabilidades claras e focadas.
Polimorfismo	RegrasJogo e EstrategiaJogador variam o comportamento em tempo de execução.

---

5. Princípios SOLID Aplicados

Princípio	Aplicação no projeto
SRP (Responsabilidade Única)	O Jogador só representa estado (nome, pontos, mão). A lógica de decisão fica na EstrategiaJogador. O fluxo do jogo fica na Partida/RegrasJogo.
OCP (Aberto /Fechado)	Carta, Baralho, RegrasJogo são abertos para extensão (novas subclasses) e fechados para modificação. Um novo jogo não altera o framework.
LSP (Substituição de Liskov)	Subclasses de Baralho e Carta substituem a classe base sem alterar o comportamento esperado. O framework trata tudo como Baralho e Carta.
ISP (Segregação de Interface)	RegrasJogo<B> define apenas os métodos necessários. Os hooks usam default methods (opcionais).
DIP (Inversão de Dependência)	A Partida depende de RegrasJogo (abstração). O Jogador depende de EstrategiaJogador (abstração). Nenhuma classe depende de implementações concretas.

---

6. Pontos de Extensão da Solução

Os pontos de extensão são os locais do código onde é possível adicionar novos comportamentos sem modificar o framework existente. O projeto possui 7+ pontos de extensão :

#	Ponto de Extensão	Como estender	Exemplo

1	Novos Baralhos	Estender `Baralho` e implementar `criarCartas()`	`BaralhoUno` (108 cartas)
2	Novas Regras	Implementar `RegrasJogo<B>`	`UnoRegras`, `PokerRegras`
3	Novas Estratégias de Jogador	Implementar `EstrategiaJogador`	`EstrategiaIAInteligente`, `EstrategiaRemota`
4	Novos Embaralhadores	Implementar `EmbaralhadorStrategy`	`EmbaralhadorInvertido`
5	Novos Listeners	Implementar `PartidaListener`	`PartidaGUListener`, `PartidaLogListener`
6	Novas Cartas	Estender `Carta`	`CartaUno` (com cor e número)
7	Novas Ações	Criar novas instâncias de `AcaoJogador`	`ACAO_COMPRAR_4` (Uno)

7.1 Componentes Reutilizáveis vs Pontos de Extensão

Componente	Tipo	Justificativa
`GameFacade`	Reutilizável	API pública genérica, usada por qualquer jogo
`Partida` / `PartidaSimples`	Reutilizável	Orquestra o fluxo genérico de qualquer jogo de cartas
`Baralho` (abstrata)	Reutilizável	Define o contrato de baralho; subclasses são pontos de extensão
`Carta` (abstrata)	Reutilizável	Define o contrato de carta; subclasses são pontos de extensão
`Jogador` / `Mao`	Reutilizável	Representação genérica de jogador e mão
`RegrasJogo<B>` (interface)	Ponto de extensão	Cada jogo implementa suas próprias regras
`EstrategiaJogador` (interface)	Ponto de extensão	Cada estratégia de decisão é uma implementação
`EmbaralhadorStrategy` (interface)	Ponto de extensão	Cada algoritmo de embaralhamento é uma implementação
`PartidaListener` (interface)	Ponto de extensão	Cada saída (console, GUI, log) é uma implementação
`BaralhoTradicional` / `BaralhoSuperTrunfo`	Aplicação	Baralhos concretos dos jogos
`BlackjackRegras` / `SuperTrunfoRegras`	Aplicação	Regras concretas dos jogos

---

8. Testes Automatizados

O projeto possui 23 testes automatizados com JUnit 4, todos passando:

Classe de Teste	Testes	Cobertura

`MaoTest`	5	Adição, remoção, limites, encapsulamento
`BaralhoTradicionalTest`	6	Criação, embaralhamento, compra, reset
`BlackjackUtilsTest`	5	Cálculo de valor, blackjack, estouro
`JogadorEstrategiaTest`	4	Estratégias, troca de estratégia
`SuperTrunfoCartaTest`	3	Atributos, comparação, super trunfo

---

9. Ambiente Utilizado

O projeto foi desenvolvido e testado no seguinte ambiente:

Item	Especificação
Processador	Intel Core i3 de 12ª geração
Memória RAM	8 GB
Sistema Operacional	Windows 11
JDK	Java 17 ou superior
Build	Maven 3.8+
Testes	JUnit 4
IDE	Visual Studio Code

9.1 Observações sobre o ambiente

- O processador Intel Core i3 de 12ª geração é suficiente para compilar e executar o projeto sem problemas, pois o framework é leve e não exige alto poder de processamento.
- Os 8 GB de RAM são mais que suficientes para rodar os testes automatizados e as aplicações cliente (Blackjack e Super Trunfo), que são aplicações de console sem interface gráfica pesada.
- O Windows 11 oferece suporte completo ao JDK 17 e ao Maven, sem necessidade de configurações especiais.
- O tempo de compilação e execução dos testes é rápido (na ordem de segundos), o que demonstra a eficiência do projeto mesmo em hardware de entrada.

---

10. Resultados e Discussões

10.1 Eficiência

O framework demonstrou alta eficiência em vários aspectos:

- Compilação rápida: o projeto compila em poucos segundos com Maven, mesmo em um notebook com processador i3 de 12ª geração e 8 GB de RAM.
- Execução leve: as aplicações cliente (Blackjack e Super Trunfo) são aplicações de console que consomem pouquíssima memória e processamento.
- Testes rápidos: os 23 testes automatizados rodam em menos de 1 segundo no total, validando o funcionamento do framework de forma ágil.
- Reutilização de código: o framework genérico (`com.map.framework`) é usado por dois jogos diferentes sem duplicação de código. Isso demonstra que a arquitetura é eficiente em termos de manutenção e desenvolvimento.

10.2 Praticidade

A praticidade do projeto se destaca em:

- API simples: a classe `GameFacade` oferece uma interface enxuta (`criarPartida` e `executarPartida`), permitindo que qualquer pessoa crie e execute uma partida com poucas linhas de código.
- Extensibilidade: criar um novo jogo é simples — basta implementar `RegrasJogo<B>`, estender `Baralho` e usar a `GameFacade`. Não é necessário entender todos os detalhes internos do framework.
- Separação de responsabilidades: a divisão em pacotes (`card`, `player`, `game`, `exception`) torna o código organizado e fácil de navegar.
- Documentação: o Javadoc completo e o relatório facilitam o entendimento e a manutenção do código por outros desenvolvedores.
- Testes automatizados: os testes garantem que alterações futuras não quebrem o funcionamento existente, aumentando a confiança no código.

10.3 Discussão

- Trade-off da tipagem genérica: o uso de generics (`Partida<B extends Baralho, R extends RegrasJogo<B>>`) aumenta a segurança de tipos, mas torna a declaração das classes um pouco mais complexa. No entanto, o benefício (erros de tipo pegos em tempo de compilação) supera a complexidade.
- Trade-off do padrão Observer: o uso de listeners adiciona uma camada de indireção, mas permite que a partida seja totalmente desacoplada da saída (console, GUI, etc.), o que é essencial para a extensibilidade.
- Limitação do ambiente: o notebook com i3 de 12ª geração e 8 GB de RAM é suficiente para este projeto, mas para jogos com interface gráfica mais pesada ou com muitos jogadores simultâneos, seria necessário um hardware mais robusto.
- Sugestões de evolução: seria possível adicionar uma interface gráfica (GUI) usando o padrão Observer (criando um `PartidaGUListener`), suporte a jogos online (via `EstrategiaJogador` remota) e novos jogos (Uno, Poker, etc.) aproveitando os pontos de extensão.

---

11. Conclusão

O projeto atende a todos os requisitos obrigatórios:

1. ✓ API pública claramente definida (`GameFacade`)
2. ✓ Pelo menos 5 pontos de extensão (7+)
3. ✓ Separação entre solução e aplicações clientes
4. ✓ Pelo menos uma aplicação cliente (2 jogos)
5. ✓ Utilização de interfaces e classes abstratas
6. ✓ Tratamento adequado de exceções
7. ✓ Encapsulamento das coleções internas
8. ✓ Testes automatizados (23 testes)
9. ✓ Documentação Javadoc da API pública
10. ✓ Diagrama de classes UML
11. ✓ Exemplos de utilização (na apresentação)

12. ✅ Justificativa das decisões de projeto (este relatório)